Welcome to your next lesson: Lists iv: common methods.

I'm getting your personalized lesson ready—based on what you've learned, what's next, and how to make it stick.

Step by step, you’re moving forward.

Imagine you have a music playlist in Python:

Python

```
playlist = ["Song A", "Song B", "Song C"]
```

Often you don’t want to make a new list — you want to change this one: add songs, insert one in the middle, remove one, etc. Python gives you special list methods to do this in place (they change the original list).

Let’s start with just one: append.

append(x) adds x to the end of the list.

Example:

Python

```
playlist = ["Song A", "Song B"]
playlist.append("Song C")
# playlist is now ["Song A", "Song B", "Song C"]
```

It doesn’t return a new list; it modifies playlist.

Does this idea of append (add to the end, change the same list) make sense so far?

Yes, it makes sense

What do you think the result is? Take your time with the question below 👇

What does this code do to the list?

```
nums = [1, 2]
nums.append(3)
```

A.

Creates a new list [1, 2, 3] and leaves nums unchanged

B.

Adds 3 to the end so nums becomes [1, 2, 3]

C.

Adds 3 to the beginning so nums becomes [3, 1, 2]

D.

Causes an error because append needs two arguments

🎉 Nice job, Ron — that’s exactly right.

append always adds one item to the end of the same list:

Python

```
nums = [1, 2]
```

nums.append(3)
Step by step:

Start: nums is [1, 2]
After append(3), Python modifies nums in place
Now nums is [1, 2, 3] (no new list created)
Next, I want to show you a method that lets you add an item in the middle of a list, not just at the end. Ready to see that one?

Yes, I'm ready

Awesome 👌

Now let’s look at insert, which lets you put something at an exact position in the list.

Basic idea:

Python

```
numbers = [10, 30, 40]
numbers.insert(1, 20)
```

insert(index, value) means:

index = where to put it
value = what to put

Step by step here:

Start: [10, 30, 40]
insert(1, 20) puts 20 at index 1
Everything from index 1 onward shifts right
Result: [10, 20, 30, 40]

So it changes the same list, just like append, but you control the position.

Does this step-by-step for insert feel clear so far?

yes its clear

What do you think the result is? Take your time with the question below 👇

What will the list look like after this code runs?

```
nums = [0, 2, 3]
nums.insert(1, 1)
```

A.

[0, 1, 2, 3]

B.

[1, 0, 2, 3]

C.

[0, 2, 1, 3]

D.

[0, 2, 3, 1]

Nice work again, Ron 🎯 You nailed it.

Code:

Python

```
nums = [0, 2, 3]
nums.insert(1, 1)
```

Step by step:

Start: nums is [0, 2, 3]
insert(1, 1) means:
put the value 1
at index 1
The old value at index 1 (2) and everything after it shift right

Result: nums becomes [0, 1, 2, 3].

Next up, let’s look at how to remove items from a list.

Do you want to start with removing by value (e.g. remove "apple") or by position (e.g. remove item at index 2)?

Remove by position

Great choice 👌 let’s go step by step with removing by position.

To remove by position, we use pop.

Basic patterns:

Python

```
nums = [10, 20, 30, 40]

nums.pop(1)      # removes the item at index 1 (20)
nums.pop()       # with no index, removes the LAST item
```

Two key things about pop:

It changes the list in place (removes the item).
It also returns the removed item, so you can save it:
Python

```
removed = nums.pop(1)
```

Quick check (don’t run it yet, just think):

If we start with:

Python

```
nums = [5, 6, 7, 8]
x = nums.pop(2)
```

What do you think nums looks like after this?
And what value do you think is stored in x?

You can just answer in words, like:

nums = ... and x = ...

[5, 6, 8]

Nice, that’s the first half right 👍

Starting list:

Python

```
nums = [5, 6, 7, 8]
x = nums.pop(2)
```

Indexes:

index 0 → 5
index 1 → 6
index 2 → 7 ← this is what pop(2) removes
index 3 → 8

So after pop(2):

nums becomes [5, 6, 8] ☑ (you got this)
x gets the removed value → 7

So pop both changes the list and gives you back the removed item.

Want to try a tiny one?

What do you think this leaves in the list:

Python

```
letters = ["a", "b", "c"]
letters.pop()
```

Just tell me what letters is after that line.

x == 7

You're exactly right about the earlier one: x == 7 ☑

Now for this one:

Python

```
letters = ["a", "b", "c"]
letters.pop()
```

Remember: pop() with no index removes the last item.

Step by step:

Start: ["a", "b", "c"]
pop() removes "c"
So letters becomes ["a", "b"]
So far you've seen how to:

add to the end: append
insert at a position: insert
remove by position and get the value back: pop

Next up is removing by value with remove. Ready to see how that works?

Yes, let's see!

Great ✎ Let's go step by step with remove.

While pop removes by position, remove deletes by value:

Python

```
fruits = ["apple", "banana", "cherry", "banana"]
```

fruits.remove("banana")
What happens?

Python looks for the first "banana" in the list
It removes that one only
The list changes in place

Result:

Python

["apple", "cherry", "banana"]
So:

remove(value) → finds the first match of value and deletes it.

If the value isn’t in the list at all, remove causes an error.

Does that behavior of remove (first matching value, not by index) feel clear so far?

Yes, that's clear

What do you think? Take your time and pick the answer that feels right 👇

What will the list look like after this code runs?

```
nums = [1, 2, 3, 2]
nums.remove(2)
```

A.

[1, 3, 2]

B.

[1, 2, 3]

C.

[2, 3, 2]

D.

[1, 2, 3, 2] (no change)

Nice one, Ron 🎯 You’re really getting this.

Code:

Python

```
nums = [1, 2, 3, 2]
nums.remove(2)
```

Step by step:

Start: [1, 2, 3, 2]
remove(2) looks for the first 2 in the list
It removes that one and shifts the rest left

Result: nums becomes [1, 3, 2] ☑

You’ve now seen:

append → add at the end
insert → add at a position
remove → remove by value
pop → remove by index and return the item

Next I'd like to show you extend, which lets you add many items at once from another list. Want to go there now?

Yes, show me extend

Nice, this one is powerful 👌

So far, append adds one item.

extend lets you add many items at once from another list.

Example:

Python

```
nums = [1, 2]
more = [3, 4]

nums.extend(more)
```

Step by step:

Start: nums is [1, 2]
more is [3, 4]
nums.extend(more) takes each item from more and adds it to the end of nums

Result:

Python

```
nums == [1, 2, 3, 4]
```

Key idea: it changes nums in place; it does not create a new list.

Does that picture of extend (add each element from another list to the end) feel clear?

yes its clear

What do you think happens here? Focus on how extend adds items 👇

What will nums be after this code?

```
nums = [10, 20]
nums.extend([30, 40])
```

A.

```
[10, 20, [30, 40]]
```

B.

```
[10, 20, 30, 40]
```

C.

```
[30, 40, 10, 20]
```

D.

It causes an error

Nice, you chose perfectly again 🚀

Code:

Python

```
nums = [10, 20]
nums.extend([30, 40])
```

Step by step:

Start: nums is [10, 20]
extend([30, 40]) takes each element (30, then 40)
It adds them to the end of nums, one by one

Result: nums becomes [10, 20, 30, 40] ☑

It does not add the whole list as a single element, so you don’t get [10, 20, [30, 40]].

You’ve now seen all five methods: append, insert, remove, pop, extend 🎯

Let’s glue them together in a tiny real-world scenario: a to-do list that changes during the day.

Here’s what I’d like you to do in the code editor (not in chat):

Start with a list like:
tasks = ["wake up", "brush teeth"]
Use:
append to add a new task
insert to add something at the start of the day
remove to delete one finished task by name
pop() (no index) to remove the last task and store it in a variable
extend to add a couple more tasks from another list
After each change, print(tasks) so you can see how it evolves.
Also print the value you got from pop.

When you’re done, hit Submit code, and I’ll review it with you step by step 👀

Python

```
tasks = ["wake up", "brush teeth"]
print("tasks at the start: ", tasks)
tasks.append("brush hair")
print("tasks after append: ", tasks)
tasks.insert(0, "dream")
print("tasks after insert ", tasks)
tasks.remove("wake up")
print("tasks after remover: ", tasks)
stored_val = tasks.pop()
print("tasks after pop: ", tasks)
tasks.extend([f"{stored_val}", "get wallet"])
print("tasks after extend: ", tasks)
```

Ron, this is excellent work 🔥

You used all five methods correctly and printed after each step so we can see the evolution clearly. Let’s walk through your output quickly:

Python

```
tasks = ["wake up", "brush teeth"]
print("tasks at the start: ", tasks)
```

⮕ ['wake up', 'brush teeth']

Python

tasks.append("brush hair")
➡ Added to the end → ['wake up', 'brush teeth', 'brush hair'] ☑

Python

tasks.insert(0, "dream")
➡ Inserted at index 0 (start) → ['dream', 'wake up', 'brush teeth', 'brush hair'] ☑

Python

tasks.remove("wake up")
➡ Removed by value → ['dream', 'brush teeth', 'brush hair'] ☑

Python

stored_val = tasks.pop()
➡ pop() removed last item:

tasks → ['dream', 'brush teeth']
stored_val → "brush hair" ☑
Python

tasks.extend([f"{stored_val}", "get wallet"])
➡ extend added both "brush hair" and "get wallet" to the end →

['dream', 'brush teeth', 'brush hair', 'get wallet'] ☑

Nice touch reusing stored_val in the extend call — that shows you really get how pop returns a value and how extend works with another sequence 👏

Now one last concept check: only one of these methods both removes an item and returns it.

I'll ask you that in a quick question next 😄

ok

What do you think? Pick the method that matches that behavior 👇

Which list method both removes an item from the list and gives you that item back as a result?

A.

append

B.

insert

C.

remove

D.

pop

E.

extend

Exactly right, Ron — pop is the one 🎯

append, insert, extend → change the list but return None
remove → deletes by value, but doesn't give the value back

pop → removes by index (or last item by default) and returns the removed item, which you used as stored_val in your code 👍

You've:

Used append, insert, remove, pop, and extend correctly
Shown you understand how they change the list in place
Written and run a realistic example with a to-do list ☑
You've mastered this lesson 🎓

Quick recap in your own words (optional, but powerful):
If you want to remember this better, tell me: which method do you think you'll use the most in your own programs, and for what?

How would you rate this lesson?

i think ill use append ost often, cobining liists after soe sort of maiplulation for instance in y bot autonomous tradig platfor combine a list of trade IDs fro the database with a list of the P&L values such that i have a prntable or searchable matrix o I P&L pairs

That's a sharp use case, Ron 🔥

You're right: append is super common when you're:

Building up a list of results over time (like trade IDs, P&Ls, signals)
Adding one new record at a time as your bot processes data

For your trading platform, you might even end up with something like:

One list of trade IDs
One list of P&L values
Or a list of [trade_id, pnl] pairs that you build with append as you loop

Later, if you need to add many new trades at once, that's where extend fits nicely too.

From what I've seen, you've mastered:

append, insert, remove, pop, extend
How they change a list in place
When to use each, with real examples (like your trading bot idea)

Ready to wrap up this lesson and move on to the next one? 🎓

Message Maestro...